Chapter 11 worked solutions

Long Context, Sparse Access, and Sequence Compression

This guide covers Exercises 1–4, 10, and 12. It follows the links between query and key positions, separates the scores used for selection from those used in attention, and counts what different designs store and read. The companion Jupyter notebook works through Exercises 5–9 and 11 and is also available as a PDF.

1. Count the grid

Number the tokens 1 through 6. Each grid has one row for the position asking a question, called the **query**, and one column for the position it can read, called the **key**. A 1 means the read is allowed and a 0 means it is blocked. For example, a 1 in row 4, column 3 lets the query at position 4 read the entry at position 3. The final column counts the allowed reads for that query.

In **full bidirectional attention**, every query may read all six keys. This includes its own position and all positions on either side.

Query / key	1	2	3	4	5	6	Allowed
1	1	1	1	1	1	1	6
2	1	1	1	1	1	1	6
3	1	1	1	1	1	1	6
4	1	1	1	1	1	1	6
5	1	1	1	1	1	1	6
6	1	1	1	1	1	1	6

There are six allowed pairs in each of six rows, for a total of $6 \cdot 6 = 36$.

In **full causal attention**, query i may read key j only when $j \leq i$. The diagonal stays open because the query can read its own position.

Query / key	1	2	3	4	5	6	Allowed
1	1	0	0	0	0	0	1
2	1	1	0	0	0	0	2
3	1	1	1	0	0	0	3
4	1	1	1	1	0	0	4
5	1	1	1	1	1	0	5
6	1	1	1	1	1	1	6

The row counts add to $1 + 2 + 3 + 4 + 5 + 6 = 21$. The decoder's last row, at position 6, can read **positions 1, 2, 3, 4, 5, and 6**. All positions in the grid are at or before that query.

In **bidirectional local attention at radius one**, query i may read key j when $|i - j| \leq 1$. Query 4 can read keys 3, 4, and 5. At the ends of the document, one neighboring position falls outside the grid.

Query / key	1	2	3	4	5	6	Allowed
1	1	1	0	0	0	0	2
2	1	1	1	0	0	0	3
3	0	1	1	1	0	0	3
4	0	0	1	1	1	0	3
5	0	0	0	1	1	1	3
6	0	0	0	0	1	1	2

The two end rows allow two reads each, and the four interior rows allow three each. The total is $2 + 4 \cdot 3 + 2 = 16$. The ones sit on the diagonal and the two adjacent diagonals. In a bidirectional local grid, a query can read the very next position as well as the previous one if both exist.

2. Trace a relay

Position 5 is **global**, so its query can read position 2 even though the positions are three steps apart. Position 9 can also read the global key at position 5, even though their distance is four. The two reads create a route across two layers.

Layer	Query that reads	Key it reads	Information now available to the query
1	5	2	Position 5 can gather information from the input at position 2.
2	9	5	Position 9 can read the representation of position 5 produced by layer 1.

Let $h_i^{(0)}$ be the input representation at position i and $h_i^{(\ell)}$ the representation after layer ℓ. The information can travel along

$$h_2^{(0)} \rightarrow h_5^{(1)} \rightarrow h_9^{(2)}.$$

All query rows in a layer read the representations from that layer's input. When position 9 reads position 5 in layer 1, it reads $h_5^{(0)}$. The updated representation $h_5^{(1)}$ becomes available as a key and value in layer 2. Both links also obey a causal mask in this particular example because each query is at or after the key it reads.

With local reads alone, each layer can move information by at most one position. A shortest route is $2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 9$. It has **seven hops**, each using a new layer. The gap is $9 - 2 = 7$, so fewer than seven local layers cannot cover it. An allowed route describes where information can flow. The learned attention and value transformations determine how much of the original signal reaches position 9.

3. Choose, then weight

Number the four positions 1 through 4. The indexer ranks them by the given scores. Position 4 has the highest score, 1.9, and position 1 has the next highest, 1.4. The selected order is therefore **position 4, then position 1**. Their zero-based Python indices are 3 and 0. Positions 2 and 3 stay outside the short list.

Selected position	Index score	Main score	Value
4	1.9	0	$(3, 0)$
1	1.4	$\log 2$	$(0, 3)$

The index scores decided which rows to gather. For the weights, take the softmax of the two **main** scores. The exponentials are $\exp(0) = 1$ and $\exp(\log 2) = 2$. Dividing each by their sum of 3 gives weights $(1/3, 2/3)$ in the selected order.

$$\frac{1}{3}(3, 0) + \frac{2}{3}(0, 3) = (1, 0) + (0, 2) = (1, 2).$$

Position 4 ranked first in selection. Once selected, it receives the smaller attention weight because its main score is lower than that of position 1. The two operations answer different questions. The indexer chooses which rows are available, and main attention decides how much each of those rows contributes.

4. Pool one coordinate

The summary takes one weighted sum across three positions. Each feature is multiplied by the weight at the same position, and the three products are added.

$$0.25 \cdot 4 + 0.5 \cdot 0 + 0.25 \cdot 2 = 1 + 0 + 0.5 = 1.5.$$

The weights sum to one, so this coordinate is a weighted average. Try the different triple $(1, 2, 1)$ with the same weights. Its contributions are $0.25 \cdot 1 = 0.25$, $0.5 \cdot 2 = 1$, and $0.25 \cdot 1 = 0.25$. They also sum to 1.5. The larger second entry offsets decreases in the first and third entries.

A query that receives only this coordinate sees 1.5 in both cases. If it also knows the weights, it can infer the constraint $x_1 + 2x_2 + x_3 = 6$ on the original entries. Many triples satisfy that constraint. The query cannot recover the exact value at any particular position from the coordinate alone, or tell whether the block's largest feature was 4 as in the first triple or 2 as in the second.

10. Account for three designs (optional)

Consider a single query with 128 visible token positions. Count a **stored KV entry** as one key and its associated value in the history used by main attention. An **index score** is a candidate score used to decide which entries to gather. A **main read** is an entry supplied to the main attention calculation. This separates the amount of history kept in memory from the amount of history consulted by either stage.

Design	Stored main-attention KV entries	Index scores	Main reads
Dense attention	128	0	128
Select eight raw entries after a full scan	128	128	8
Read all stride-eight summaries and the four recent raw entries	20	0	20

Dense attention retains and reads every raw entry. It computes its main scores directly, so there is no separate indexer. The selection design also retains all 128 raw entries because any of them might be selected. A full scan assigns one index score to each raw entry. Only the eight chosen entries are passed to main attention.

With stride eight, the 128 visible positions give $128/8 = 16$ completed summaries. The design keeps those summaries and the four most recent raw entries, for $16 + 4 = 20$ stored entries. If positions are numbered 1 to 128, the last summary covers positions 121 through 128 and the raw entries cover 125 through 128. The summary occupies its own cache entry and the four raw entries occupy four more. All 20 go into main attention, so a separate ranking scan is unnecessary.

These counts follow the chapter's table for one query. Actual storage in bytes also depends on the key and value widths and their data types. An indexer's own keys and scratch buffers have separate storage, and computing the summaries requires work before the query reads them.

12. Plan a long-document comparison (optional)

Imagine a registry report with many shipment IDs, archive IDs, and seal codes. Near the end, the reader asks for the seal code of shipment R17. In one version of the report, an early record says *Shipment R17 uses seal code C42*. In the other version, one record says *Shipment R17 was sent to archive A03*, and a separate record says *Archive A03 uses seal code C42*. The first task requires finding one exact value. The second requires finding the archive for the shipment and following that reference to a second record. Ask for the archive ID and seal code together in the second task, so a response with just one correct field receives partial credit only.

Build documents at 2,048, 8,192, and 32,768 tokens with a common tokenizer. Put the question at the end of each document and count positions after tokenization. For a document of length T, place the required record in the one-fact task at token $T/8$. In the two-fact task, start the two records at $T/8$ and $3T/8$ and counterbalance which record appears first. For a nearby control, place the records at $T - 96$ and $T - 64$, or just at $T - 64$ for the one-fact task. Use short, fixed-length record templates so the records fit before the final question without overlapping. Record their actual start and end positions after tokenization.

Fill the intervening passages with plausible distractors, such as shipments sent to other archives and archives with other seal codes. Make the requested answer unique in each document. Randomly assign IDs and codes so that the question, the distance, and the surrounding filler do not predict the answer. Create paired near and far versions of each document by moving the relevant records and using the same question and answer. Split by underlying document before generating variants, so a near version cannot be used for training while its far counterpart appears in the test set. Reserve 400 test documents per task and length, each with both placements. Give every system the same prompts, distractors, and answer instructions.

Use causal decoders with the same model width, number of layers, tokenizer, position convention, and output head. Use a context limit of at least 32,800 tokens to leave room for the answer and the timing continuation. Start shared parameters from paired initializations and use the same training and validation documents and number of training tokens. One concrete comparison uses eight layers and the following access policies.

Method	History available to one query
Dense attention	All valid current and earlier raw positions
Local windows	The current position and up to 128 earlier positions in each layer, with no global positions
Learned selection	Up to 128 raw positions, chosen by a learned index score from a scan of all causally eligible raw positions
Sequence summaries	One summary per completed block of 16 positions, plus the 128 most recent raw entries

For selection, rank only eligible positions, then compute the main attention weights over the gathered entries. Train the index scorer on the training documents using an auxiliary ranking loss that favors the annotated evidence positions for the answer query. Give all four models separate prompts during training that ask for the same annotated supporting record IDs, and report the selector's additional objective and training cost as part of its design. For summaries, a query can read a block only once its last position is visible. Keep the raw and summary entries separate when a recent position is represented in both. A local stack can relay information beyond a single layer's window, but with eight layers of radius 128 a path can move at most $8 \cdot 128 = 1,024$ positions toward the final query. The near and far placements therefore test distinct access conditions. Freeze the settings on validation data before scoring the test set.

For the quality test, let every model generate greedily. Ask for one line in the fixed format `seal=CODE` or `archive=ID seal=CODE`, and stop at a newline or after 32 tokens. Measure quality with exact match on the normalized seal code for the one-fact task. For the two-fact task, report accuracy for each field and joint accuracy, which requires both the archive ID and the seal code. Trim surrounding whitespace and fold case before comparison. Treat missing or malformed fields as incorrect. Report accuracy by task, context length, and evidence placement, with paired differences on the same documents. If estimating uncertainty, resample entire underlying documents so their near and far versions stay paired. As a diagnostic for the learned selector, record whether any key from each required record appears in the selected set at the answer query for a specified layer and head. This measures direct access there. The accuracy metric measures what the model produces after all layers.

Measure memory and time on the same GPU with float16 inference and batch size one. Record the device model, software, and precision actually used by each kernel. At each length, record persistent cache bytes after prefill and after decoding. Count the actual arrays from their shapes and dtype sizes across all layers. Include stored raw keys and values, summary entries, any persistent partial-summary buffers, and persistent index keys and position metadata. Report those components separately, as well as the allocated capacity if it exceeds the live payload. Measure peak temporary and total memory separately so large score grids, index scans, and gather buffers remain visible. Log the number of index scores and main reads separately because a small gathered set can still require a full index scan.

Time the prompt pass from device-ready input IDs through the logits for the first continuation token. Include any work required to build the raw cache, summaries, and index state. For the following steps, use the same replayed continuation of 32 tokens for every method. The prompt supplies the first prediction, so feed the first 31 continuation IDs and time those 31 incremental calls. Include cache updates, selection, gathering, and attention in the decode measurement. Warm up the methods, synchronize the device at the boundaries of each measurement, and run 20 trials per method on the same prompts, rotating the order. Report median prefill latency and median decode milliseconds per step. If a length exceeds the memory limit for a method, report that outcome at that length. Compare latency across the methods that complete the same workload.